

Samukelo Magagula 230485731

Computer Graphics Game

Introduction

In my game, I created a version of Subway Surfers using WebGL, JavaScript, CSS, and HTML with the help of WebGL Fundamentals. My game is played using arrow keys and it was tested using Microsoft Edge and it runs. The challenges I encountered were a lot as I had only 3 months of experience with WebGL. Here are some of the challenges I encountered:

Designing a game like Subway Surfers using WebGL presents a number of challenges, especially for a university student with only three months of WebGL knowledge. Here are some of the primary challenges I encountered:

1. Understanding and Implementing Basic WebGL Concepts:

- Shaders: Writing vertex and fragment shaders, understanding GLSL (OpenGL Shading Language), and managing shader programs can be complex.
- Buffers and Attributes: Setting up vertex buffers, element buffers, and attribute pointers for rendering objects in the scene.

2. Creating and Managing Complex 3D Models:

- Model Loading: Loading and rendering complex 3D models (e.g., characters, trains, obstacles) from external files via script files.
- Texture Mapping: Applying textures correctly
- Animation: Implementing animation for characters to move

3. Camera Movement and Controls:

- Third-Person Camera: Implementing a camera that follows the player smoothly, including handling camera rotation and perspective projection.
- User Input: Capturing and responding to user input for character movement and interaction with the game environment.

4. Collision Detection and Response:

- Colliding- implementing a collision algorithm to ensure that if my player touches any unwanted barrier its game over

5. Performance Optimization:

- Rendering Efficiency: Reducing draw calls, optimizing shaders, and managing resources to maintain a high frame rate.

6. Game Mechanics and Logic:

- Game State Management: Managing different game states (e.g., playing, game over, etc.) and transitions between them.

7. Debugging and Testing:

- Debugging WebGL Code: Identifying and fixing issues in WebGL code, which was challenging due to the complexity of graphics programming.
- Browser Compatibility: Ensuring that the game works consistently across different browsers and devices.

What doesn't work in my Game

When my player crushes into certain barriers in the game it does not display game over instead it continues, but when it crushes into the train it displays game over. There is a mistake in my collision detection function and when I ask ChatGPT to fix the error it creates more errors. I tested this game on Microsoft Edge and Firefox but on Chrome it tends not to run on some PC.

How I addressed these challenges I had:

1. Through Learning Resources: I used online tutorials, documentation, and forums to understand WebGL concepts better.
2. Use of YouTube: I watched a guy code subway surf using Three.js and I used that logic he used to do it in WebGL
3. Use of AI's: Through the helps of AI's like ChatGPT and Copilot it simplified debugging for me and also assisted me throughout coding the whole game
4. Collaboration: Working with peers and people who have been exposed to WebGL in the past
5. Iterative Testing: Continuously test the game on the browser Microsoft edge to identify and resolve performance and compatibility issues early.

Explanation of how I designed my game

Firstly, I used the `gl-matrix.js` library, `jquery-3.3.1.min.js` library and `webg-utils.js` library. Then I created several JavaScript files, the most important file being the `main.js` initializes and runs a basic 3D game using WebGL, inspired by Subway Surfers. It sets up WebGL context, compiles shaders, and initializes various buffers for 3D objects such as the player, ground, tracks, walls, coins, obstacles, barriers, and boosts. The vertex and fragment shaders handle lighting, texture application, and optional grayscale rendering. The ``main`` function creates and manages these objects, sets their positions, and handles game logic such as player movement, collision detection, and game states like start, game over, and finish. The ``render`` function, called repeatedly using ``requestAnimationFrame``, updates the scene based on game events, random object generation, and user input, ensuring the game runs smoothly with features like flashing effects and score updates. The `draw.js` clears the WebGL scene, sets up perspective and model-view matrices, and draws 3D objects with applied textures and lighting. The ``clearScene`` function configures the background color, depth testing, and creates a perspective projection matrix. The ``drawScene`` function prepares buffers for positions, texture coordinates, and normal, binds textures, sets shader uniforms, and renders the object using the configured shaders. The scene is updated continuously to reflect object transformations and lighting changes. The ``barrier`` function creates a 3D barrier object with vertices, indices, normals, and textures, positioned on a specified track. The ``barrier_delete`` function replaces an old barrier with a new one at a random track and initial position. The ``barrier_tick`` function updates the barriers' positions, checks for collisions with the player, and adjusts the game's speed accordingly, resetting barriers that move out of view. The ``boost`` function creates a 3D boost object with vertices, indices, normals, textures, and a random type (jump or fly), while ``boost_tick`` updates the boosts' positions, checks for player collisions to activate boosts, and removes or replaces boosts as needed. The JavaScript file `camera.js` defines a ``create_camera`` function that generates a view matrix for a camera positioned at a computed location on a circle and aimed at a specified target point. It utilizes the ``lookAt`` function to calculate the camera matrix based on the camera's position, target point, and up vector. The resulting view matrix is inverted and returned for use in rendering 3D scenes. Then the application of the other JavaScript files is similar to the `barrier.js` file. The HTML file sets up a webpage for a game called "Subway Surfers", including a canvas element for rendering graphics. It also incorporates modal boxes for displaying game information and score, with JavaScript functions handling game start, restart, and game over events. Additionally, it links several JavaScript files for game logic and rendering, along with external libraries like jQuery and WebGL utilities. The CSS styles define the layout and appearance of elements in the game interface

Credits

1. Mitch Williams- https://youtu.be/raErq7oDIgY?si=d8_aGijhPvu5xYh3

The videos were instrumental in helping me grasp the process of prompting ChatGPT effectively, as well as mastering techniques for rendering graphics and debugging.

2. [WebGL Fundamentals](#)
3. [dom-examples/webgl-examples at main · mdn/dom-examples \(github.com\)](#)
4. ChatGPT and Copilot which are AI's which assisted me